



## 1 Descripción General

El objetivo de este proyecto es desarrollar un **front-end de compilador** para un subconjunto de un lenguaje tipo C utilizando herramientas clásicas de construcción de compiladores como Flex y Bison (o Lex/Yacc).

El sistema deberá ser capaz de:

- realizar análisis léxico,
- análisis sintáctico,
- análisis semántico,
- manejo de tablas de símbolos,
- manejo de tabla de tipos,
- verificación de tipos,
- manejo de ámbitos,
- y generación de código intermedio de tres direcciones.

El compilador **NO** deberá generar código ensamblador ni ejecutables.

El resultado final será la generación de **código intermedio (TAC — Three Address Code)** junto con la validación semántica del programa fuente.

El proyecto deberá implementarse en equipos de hasta 4 integrantes.

## 2 Objetivos Académicos

El proyecto busca que el estudiante comprenda e implemente:

- construcción de analizadores léxicos,
- diseño de gramáticas libres de contexto,
- parsing LR mediante Yacc/Bison,
- traducción dirigida por sintaxis,
- tablas de símbolos,
- tablas de tipos,
- verificación semántica,
- control de ámbitos,
- generación de temporales y etiquetas,
- y generación de código intermedio.

## 3 Lenguaje a Implementar

El compilador deberá soportar un subconjunto de un lenguaje tipo C con las siguientes características.

### 3.1 Tipos básicos

- int
- float
- char
- bool

### 3.2 Estructuras soportadas

- arreglos multidimensionales,
- structs,
- funciones,
- variables locales y globales,
- parámetros,
- llamadas a función.

### 3.3 Sentencias

- asignaciones,
- if,
- if-else,
- while,
- for,
- break,
- return.

### 3.4 Expresiones

Operadores soportados:

- aritméticos:

– +

– -

– \*

– /

– %

- relacionales:

– i

– <

– i=

– <=

– ==

– !=

- lógicos:

– &&

– \_\_\_\_\_

## **4 Requerimientos Mínimos**

### **4.1 1. Analizador Léxico**

Implementado con Flex/Lex.

Debe reconocer:

- identificadores,
- palabras reservadas,
- literales,
- operadores,
- delimitadores,
- comentarios,
- espacios en blanco.

Debe reportar errores léxicos.

### **4.2 2. Analizador Sintáctico**

Implementado con Bison/Yacc.

Debe construirse a partir de una gramática libre de contexto.

Debe detectar y reportar errores sintácticos.

### **4.3 3. Análisis Semántico**

El compilador deberá verificar:

- variables duplicadas,
- funciones duplicadas,
- structs duplicados,
- identificadores no declarados,
- compatibilidad de tipos,
- parámetros de funciones,
- tipo de retorno,
- indexación válida de arreglos,
- acceso correcto a structs,
- uso válido de operadores.

### **4.4 4. Tabla de Símbolos**

Debe manejar:

- ámbitos anidados,
- variables,
- funciones,
- parámetros,
- structs.

## 4.5 5. Tabla de Tipos

Debe permitir:

- tipos básicos,
- arreglos,
- structs,
- tipos compuestos,
- tamaños de tipos,
- tipos base.

## 4.6 6. Generación de Código Intermedio

El compilador deberá generar código intermedio de tres direcciones.

Ejemplo:

```
a = b + c * d;
```

Posible salida:

```
t1 = c * d
t2 = b + t1
a = t2
```

Debe incluir:

- temporales,
- etiquetas,
- saltos condicionales,
- llamadas a función,
- parámetros,
- returns.

## 5 Uso de Inteligencia Artificial

El uso de herramientas de Inteligencia Artificial está permitido.

Esto incluye:

- asistentes de programación,
- generación de código,
- depuración,
- documentación,
- explicación de errores,
- generación de pruebas.

El enfoque del proyecto permite el uso de metodologías de desarrollo asistidas por IA o *vibe coding*. Sin embargo:

- el equipo debe comprender completamente su implementación,
- todos los integrantes deben poder explicar el funcionamiento del sistema,

- y deberán defender técnicamente las decisiones tomadas.

Durante la evaluación se podrá solicitar:

- explicación de reglas semánticas,
- demostraciones,
- modificaciones en vivo,
- corrección de errores,
- o extensión de funcionalidades.

## **6 Entregables**

### **6.1 Código Fuente Completo**

Repositorio en GitHub con:

- analizador léxico,
- analizador sintáctico,
- estructuras auxiliares,
- tabla de símbolos,
- tabla de tipos,
- generador de TAC,
- casos de prueba.

El repositorio deberá incluir instrucciones de compilación y ejecución.

### **6.2 Documento Técnico**

Documento en PDF que incluya:

#### **Portada**

- nombre del proyecto,
- integrantes,
- grupo,
- fecha.

#### **Descripción del lenguaje**

- tokens,
- sintaxis,
- características soportadas.

#### **Gramática**

- gramática utilizada,
- explicación de conflictos si existen,
- precedencias y asociatividades.

## **Análisis Semántico**

Descripción de:

- atributos,
- reglas semánticas,
- manejo de ámbitos,
- manejo de tipos,
- verificación semántica.

## **Tablas**

Descripción de:

- tabla de símbolos,
- tabla de tipos,
- temporales,
- etiquetas.

## **Casos de prueba**

Incluir:

- programas válidos,
- errores léxicos,
- errores sintácticos,
- errores semánticos.

## **6.3 Casos de Prueba**

El repositorio deberá incluir una carpeta con programas de prueba.

Mínimo:

- 5 programas válidos,
- 3 programas con errores semánticos,
- 3 programas con errores léxicos,
- 3 programas con errores sintácticos.

## **6.4 Ejecución**

El proyecto deberá poder ejecutarse desde línea de comandos.

Ejemplo:

```
./compiler archivo.txt
```

Salida esperada:

- errores encontrados,
- o código intermedio generado.

## 6.5 Exposición y Defensa

Todos los integrantes deberán participar en la exposición.

Se evaluará:

- comprensión del proyecto,
- claridad técnica,
- correctitud del compilador,
- calidad del código,
- calidad de pruebas,
- robustez semántica.

## 7 Criterios de Evaluación

| Criterio                  | Porcentaje |
|---------------------------|------------|
| Análisis léxico           | 10%        |
| Análisis sintáctico       | 15%        |
| Análisis semántico        | 30%        |
| Tabla de símbolos y tipos | 15%        |
| Generación de TAC         | 20%        |
| Documentación y pruebas   | 10%        |

## 8 Recomendaciones

Se recomienda desarrollar el proyecto por etapas:

1. Léxico.
2. Sintaxis.
3. Tabla de símbolos.
4. Tipos.
5. Semántica.
6. Pruebas.

También se recomienda mantener:

- commits frecuentes,
- pruebas automatizadas,
- ejemplos pequeños,
- y documentación continua.

## 9 Estructuras Auxiliares

### 9.1 Tabla de Símbolos

Cada entrada contiene:

*(nombre, direccion, tipo, categoria, argumentos)*

Categorías válidas:

- var
- arg
- func
- struct
- temp

### 9.2 Tabla de Tipos

Cada tipo almacena:

*(id, nombre, tamaño, tipoBase, tablaInterna)*

### 9.3 Pilas Utilizadas

- PilaTS
- PilaOffset
- PilaBreak
- PilaReturn
- PilaTrue
- PilaFalse
- PilaNext

### 9.4 Variables Globales

- dir
- tipoFuncionActual

| PRODUCCIÓN                                  | REGLAS SEMÁNTICAS                                                                                            |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| $P \rightarrow H$                           | $dir \leftarrow 0$<br>$PilaTs.push(nuevaTs())$                                                               |
| $H \rightarrow D H$                         |                                                                                                              |
| $H \rightarrow \epsilon$                    |                                                                                                              |
| $D \rightarrow T L ;$                       | $L.tipo = T.tipo$                                                                                            |
| $D \rightarrow \text{struct id } \{ D \} ;$ | <b>Si no</b> $PilaTs.bottom().existe(id)$ <b>Entonces</b><br>$PilaTs.push(nuevaTs())$<br>$PilaDir.push(dir)$ |

|                                                       |                                                                                                                                                                                                                                                               |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                       | <pre> dir ← 0 *ts = PilaTs.top() PilaTs.pop() T.tipo = tablaTipos.add(id,'struct',*ts) PilaTs.bottom().add(id,-1,T.tipo,'struct',nulo) dir ← PilaDir.top() PilaDir.pop() </pre>                                                                               |
|                                                       | <b>Sino</b>                                                                                                                                                                                                                                                   |
|                                                       | error(Estructura duplicada)                                                                                                                                                                                                                                   |
|                                                       | <b>Fin Si</b>                                                                                                                                                                                                                                                 |
| $D \rightarrow \text{def } T \text{ id}(F) \{ D R \}$ | <pre> PilaLabelNext.push(newLabel()) </pre>                                                                                                                                                                                                                   |
|                                                       | <b>Si no</b> PilaTs.bottom().existe(id) <b>Entonces</b>                                                                                                                                                                                                       |
|                                                       | <pre> PilaTs.push(nuevaTs()) PilaDir.push(dir) dir ← 0 tipoFuncActual ← T.tipo label(id) S.code label(PilaLabelNext.top()) PilaTs.pop() PilaTs.bottom().add(id,-1,T.tipo,'func',F.lista) dir ← PilaDir.top() PilaDir.pop() </pre>                             |
|                                                       | <b>Sino</b>                                                                                                                                                                                                                                                   |
|                                                       | error(Función duplicada)                                                                                                                                                                                                                                      |
|                                                       | <b>Fin Si</b>                                                                                                                                                                                                                                                 |
|                                                       | PilaLabelNext.pop()                                                                                                                                                                                                                                           |
| $T \rightarrow B A$                                   | <pre> T.tipo = A.tipo A.base = B.tipo </pre>                                                                                                                                                                                                                  |
| $B \rightarrow \text{int}$                            | B.tipo = tablaTipos.getId('int')                                                                                                                                                                                                                              |
| $B \rightarrow \text{float}$                          | B.tipo = tablaTipos.getId('float')                                                                                                                                                                                                                            |
| $B \rightarrow \text{bool}$                           | B.tipo = tablaTipos.getId('bool')                                                                                                                                                                                                                             |
| $B \rightarrow \text{char}$                           | B.tipo = tablaTipos.getId('char')                                                                                                                                                                                                                             |
| $B \rightarrow \text{id}$                             | <b>Si</b> tablaTipos.existe(id) <b>Entonces</b><br>B.tipo = tablaTipos.getId(id)                                                                                                                                                                              |
|                                                       | <b>Sino</b>                                                                                                                                                                                                                                                   |
|                                                       | error(Struct no declarado)                                                                                                                                                                                                                                    |
|                                                       | <b>Fin Si</b>                                                                                                                                                                                                                                                 |
| $A \rightarrow [ \text{num} ] A_1$                    | <b>Si</b> num.tipo = int <b>Entonces</b><br><b>Si</b> num.val > 0 <b>Entonces</b><br>A.tipo = tablaTipos.add(num.val,A <sub>1</sub> .tipo,'array')<br>A <sub>1</sub> .base = A.base<br><b>Sino</b><br>error(El índice debe ser mayor a cero)<br><b>Fin Si</b> |
|                                                       | <b>Sino</b>                                                                                                                                                                                                                                                   |
|                                                       | error(El índice debe ser entero)                                                                                                                                                                                                                              |
|                                                       | <b>Fin Si</b>                                                                                                                                                                                                                                                 |
| $A \rightarrow \varepsilon$                           | A.tipo = A.base                                                                                                                                                                                                                                               |

|                                   |                                                                                                                                                                                                                                                                                                 |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $L \rightarrow L_1, \text{id}$    | $L_1.\text{tipo} = L.\text{tipo}$<br><b>Si no</b> PilaTs.top().existe(id) <b>Entonces</b><br>PilaTs.top().add(id, dir, L.tipo, 'var')<br>$\text{dir} \leftarrow \text{dir} + \text{tablaTipos.getTam}(L.\text{tipo})$<br><b>Sino</b><br>error(Variable duplicada)<br><b>Fin Si</b>              |
| $L \rightarrow \text{id}$         | <b>Si no</b> PilaTs.top().existe(id) <b>Entonces</b><br>PilaTs.top().add(id, dir, L.tipo, 'var')<br>$\text{dir} \leftarrow \text{dir} + \text{tablaTipos.getTam}(L.\text{tipo})$<br><b>Sino</b><br>error(Variable duplicada)<br><b>Fin Si</b>                                                   |
| $F \rightarrow G$                 | F.lista = G.lista                                                                                                                                                                                                                                                                               |
| $F \rightarrow \varepsilon$       | F.lista = nuevaLista()                                                                                                                                                                                                                                                                          |
| $G \rightarrow G_1, T \text{id}$  | <b>Si no</b> PilaTs.top().existe(id) <b>Entonces</b><br>PilaTs.top().add(id, dir, T.tipo, 'arg')<br>$\text{dir} \leftarrow \text{dir} + \text{tablaTipos.getTam}(T.\text{tipo})$<br><b>Sino</b><br>error(Parámetro duplicado)<br><b>Fin Si</b><br>G.lista = $G_1$ .lista<br>G.lista.add(T.tipo) |
| $G \rightarrow T \text{id}$       | <b>Si no</b> PilaTs.top().existe(id) <b>Entonces</b><br>PilaTs.top().add(id, dir, T.tipo, 'arg')<br>$\text{dir} \leftarrow \text{dir} + \text{tablaTipos.getTam}(T.\text{tipo})$<br><b>Sino</b><br>error(Parámetro duplicado)<br><b>Fin Si</b><br>G.lista = nuevaLista()<br>G.lista.add(T.tipo) |
| $R \rightarrow S R$               | PilaLabelNext.push(newLabel())<br>label(PilaLabelNext.top())<br>PilaLabelNext.pop()                                                                                                                                                                                                             |
| $R \rightarrow \varepsilon$       |                                                                                                                                                                                                                                                                                                 |
| $S \rightarrow \text{break};$     | genCode('goto' PilaLabelBreak.top())                                                                                                                                                                                                                                                            |
| $S \rightarrow \text{return } E;$ | <b>Si</b> tipoFuncActual = E.tipo <b>Entonces</b><br>genCode('return' E.dir)<br><b>Sino</b><br>error(Tipo de retorno incorrecto)<br><b>Fin Si</b>                                                                                                                                               |
| $S \rightarrow \text{id} = E;$    | <b>Si</b> PilaTs.top().existe(id) <b>Entonces</b><br>tipo = PilaTs.top().getTipo(id)<br><b>Sino Si</b> PilaTs.bottom().existe(id) <b>Entonces</b><br>tipo = PilaTs.bottom().getTipo(id)<br><b>Sino</b><br>error(El id no fue declarado)<br><b>Fin Si</b>                                        |

|                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      | <b>Si</b> compatibles(tipo,E.tipo) <b>Entonces</b><br>$\alpha_1 = \text{reducir}(E.\text{dir}, E.\text{tipo}, \text{tipo})$<br>genCode(id '=' $\alpha_1$ )<br><b>Sino</b><br>error(Tipos incompatibles)<br><b>Fin Si</b>                                                                                                                                                                                                                              |
| $S \rightarrow C = E ;$                              | <b>Si</b> compatibles(C.tipo, E.tipo) <b>Entonces</b><br>$\alpha_1 = \text{reducir}(E.\text{dir}, E.\text{tipo}, C.\text{tipo})$<br>genCode(C.base '['C.dir']' '=' $\alpha_1$ )<br><b>Sino</b><br>error(tipos incompatibles)<br><b>Fin Si</b>                                                                                                                                                                                                         |
| $S \rightarrow Z = E ;$                              | <b>Si</b> compatibles(Z.tipo, E.tipo) <b>Entonces</b><br>$\alpha_1 = \text{reducir}(E.\text{dir}, E.\text{ipo}, Z.\text{tipo})$<br>genCode(Z.base '['Z.dir']' '=' $\alpha_1$ )<br><b>Sino</b><br>error(tipos incompatibles)<br><b>Fin Si</b>                                                                                                                                                                                                          |
| $S \rightarrow \text{if}( E ) S_1$                   | PilaLabelTrue.push(newLabel())<br>PilaLabelFalse.push(newLabel())<br>genCode('if' E.dir 'goto' PilaLabelTrue.top())<br>genCode('goto' PilaLabelFalse.top())<br>label(PilaLabelTrue.top())<br>$S_1.\text{code}$<br>label(PilaLabelFalse.top())<br>PilaLabelTrue.pop()<br>PilaLabelFalse.pop()                                                                                                                                                          |
| $S \rightarrow \text{if}( E ) S_1 \text{ else } S_2$ | PilaLabelTrue.push(newLabel())<br>PilaLabelFalse.push(newLabel())<br>PilaLabelNext.push(newLabel())<br>genCode('if' E.dir 'goto' PilaLabelTrue.top())<br>genCode('goto' PilaLabelFalse.top())<br>label(PilaLabelTrue.top())<br>$S_1.\text{code}$<br>genCode('goto' PilaLabelNext.top())<br>label(PilaLabelFalse.top())<br>$S_2.\text{code}$<br>label(PilaLabelNext.top())<br>PilaLabelTrue.pop()<br>PilaLabelFalse.pop()<br>PilaLabelNext.pop()       |
| $S \rightarrow \text{while}( E ) S_1$                | PilaBreak.push(PilaLabelNext.top())<br>PilaLabelNext.push(newLabel())<br>PilaBreak.push(PilaLabelNext.top())<br>PilaLabelTrue.push(newLabel())<br>PilaLabelFalse.push(newLabel())<br>label(PilaLabelNext.top())<br>$E.\text{code}$<br>genCode('if' E.dir 'goto' PilaLabelTrue.top())<br>genCode('goto' PilaLabelFalse.top())<br>label(PilaLabelTrue.top())<br>$S_1.\text{code}$<br>genCode('goto' PilaLabelNext.top())<br>label(PilaLabelFalse.top()) |

|                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                             | PilaLabelTrue.pop()<br>PilaLabelFalse.pop()<br>PilaLabelNext.pop()                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| $S \rightarrow \text{for}(S_1; E; S_2) S_3$ | PilaBreak.push(PilaLabelNext.top())<br>PilaLabelNext.push(newLabel())<br>PilaLabelTrue.push(newLabel())<br>PilaLabelFalse.push(newLabel())<br>PilaLabelInc.push(newLabel())<br>$S_1$ .code<br>label(PilaLabelNext.top())<br>$E$ .code<br>genCode('if' $E$ .dir 'goto' PilaLabelTrue.top())<br>genCode('goto' PilaLabelFalse.top())<br>label(PilaLabelInc.top())<br>$S_2$ .code<br>genCode('goto' PilaLabelNext.top())<br>label(PilaLabelTrue.top())<br>$S_3$ .code<br>genCode('goto' PilaLabelInc.top())<br>label(PilaLabelFalse.top())<br>PilaLabelTrue.pop()<br>PilaLabelFalse.pop()<br>PilaLabelNext.pop()<br>PilaLabelInc.pop()<br>PilaBreak.pop() |
| $E \rightarrow E_1 + E_2$                   | <b>Si</b> compatibles( $E_1$ .tipo, $E_2$ .tipo) <b>Entonces</b><br>$E$ .dir = nuevaTemporal()<br>$E$ .tipo = max( $E_1$ .tipo, $E_2$ .tipo)<br>$\alpha_1$ = ampliar( $E_1$ .dir, $E_1$ .tipo, $E$ .tipo)<br>$\alpha_2$ = ampliar( $E_2$ .dir, $E_2$ .tipo, $E$ .tipo)<br>genCode( $E$ .dir '=' $\alpha_1$ '+' $\alpha_2$ )<br><b>Sino</b><br>error(Tipos incompatibles)<br><b>Fin Si</b>                                                                                                                                                                                                                                                              |
| $E \rightarrow E_1 - E_2$                   | Igual que suma                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| $E \rightarrow E_1 * E_2$                   | Igual que suma                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| $E \rightarrow E_1 / E_2$                   | Igual que suma                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| $E \rightarrow E_1 \% E_2$                  | <b>Si</b> $E_1$ .tipo = int <b>y</b> $E_2$ .tipo = int <b>Entonces</b><br>$E$ .dir = nuevaTemporal()<br>$E$ .tipo = int<br>genCode( $E$ .dir '=' $E_1$ .dir '%' $E_2$ .dir)<br><b>Sino</b><br>error(El operador módulo requiere enteros)<br><b>Fin Si</b>                                                                                                                                                                                                                                                                                                                                                                                              |
| $E \rightarrow E_1    E_2$                  | <b>Si</b> $E_1$ .tipo = $E_2$ .tipo <b>y</b> $E_1$ .tipo = bool <b>Entonces</b><br>$E$ .dir = nuevaTemporal()<br>$E$ .tipo = bool<br>tipo = max( $E_1$ .tipo, $E_2$ .tipo)<br>$\alpha_1$ = ampliar( $E_1$ .dir, $E_1$ .tipo, tipo)<br>$\alpha_2$ = ampliar( $E_2$ .dir, $E_2$ .tipo, tipo)<br>genCode( $E$ .dir '=' $\alpha_1$ '  ' $\alpha_2$ )<br><b>Sino</b>                                                                                                                                                                                                                                                                                        |

|                                  |                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                  | error(tipos incompatibles)                                                                                                                                                                                                                                                                                                                                             |
|                                  | <b>Fin Si</b>                                                                                                                                                                                                                                                                                                                                                          |
| $E \rightarrow E_1 \ \&\& \ E_2$ | <b>Si</b> $E_1.tipo = E_2.tipo$ <b>y</b> $E_1.tipo = \text{bool}$ <b>Entonces</b><br>$E.dir = \text{nuevaTemporal}()$<br>$E.tipo = \text{bool}$<br>$tipo = \max(E_1.tipo, E_2.tipo)$<br>$\alpha_1 = \text{ampliar}(E_1.dir, E_1.tipo, tipo)$<br>$\alpha_2 = \text{ampliar}(E_2.dir, E_2.tipo, tipo)$<br>$\text{genCode}(E.dir \ '=\ ' \alpha_1 \ ' \&\& \ ' \alpha_2)$ |
|                                  | <b>Sino</b><br>error(tipos incompatibles)                                                                                                                                                                                                                                                                                                                              |
|                                  | <b>Fin Si</b>                                                                                                                                                                                                                                                                                                                                                          |
| $E \rightarrow E_1 > E_2$        | <b>Si</b> $\text{compatibles}(E_1.tipo, E_2.tipo)$ <b>Entonces</b><br>$E.dir = \text{nuevaTemporal}()$<br>$E.tipo = \text{bool}$<br>$tipo = \max(E_1.tipo, E_2.tipo)$<br>$\alpha_1 = \text{ampliar}(E_1.dir, E_1.tipo, tipo)$<br>$\alpha_2 = \text{ampliar}(E_2.dir, E_2.tipo, tipo)$<br>$\text{genCode}(E.dir \ '=\ ' \alpha_1 \ '> \ ' \alpha_2)$                    |
|                                  | <b>Sino</b><br>error(tipos incompatibles)                                                                                                                                                                                                                                                                                                                              |
|                                  | <b>Fin Si</b>                                                                                                                                                                                                                                                                                                                                                          |
| $E \rightarrow E_1 < E_2$        | <b>Si</b> $\text{compatibles}(E_1.tipo, E_2.tipo)$ <b>Entonces</b><br>$E.dir = \text{nuevaTemporal}()$<br>$E.tipo = \text{bool}$<br>$tipo = \max(E_1.tipo, E_2.tipo)$<br>$\alpha_1 = \text{ampliar}(E_1.dir, E_1.tipo, tipo)$<br>$\alpha_2 = \text{ampliar}(E_2.dir, E_2.tipo, tipo)$<br>$\text{genCode}(E.dir \ '=\ ' \alpha_1 \ '< \ ' \alpha_2)$                    |
|                                  | <b>Sino</b><br>error(tipos incompatibles)                                                                                                                                                                                                                                                                                                                              |
|                                  | <b>Fin Si</b>                                                                                                                                                                                                                                                                                                                                                          |
| $E \rightarrow E_1 \geq E_2$     | <b>Si</b> $\text{compatibles}(E_1.tipo, E_2.tipo)$ <b>Entonces</b><br>$E.dir = \text{nuevaTemporal}()$<br>$E.tipo = \text{bool}$<br>$tipo = \max(E_1.tipo, E_2.tipo)$<br>$\alpha_1 = \text{ampliar}(E_1.dir, E_1.tipo, tipo)$<br>$\alpha_2 = \text{ampliar}(E_2.dir, E_2.tipo, tipo)$<br>$\text{genCode}(E.dir \ '=\ ' \alpha_1 \ '\geq \ ' \alpha_2)$                 |
|                                  | <b>Sino</b><br>error(tipos incompatibles)                                                                                                                                                                                                                                                                                                                              |
|                                  | <b>Fin Si</b>                                                                                                                                                                                                                                                                                                                                                          |
| $E \rightarrow E_1 \leq E_2$     | <b>Si</b> $\text{compatibles}(E_1.tipo, E_2.tipo)$ <b>Entonces</b><br>$E.dir = \text{nuevaTemporal}()$<br>$E.tipo = \text{bool}$<br>$tipo = \max(E_1.tipo, E_2.tipo)$<br>$\alpha_1 = \text{ampliar}(E_1.dir, E_1.tipo, tipo)$<br>$\alpha_2 = \text{ampliar}(E_2.dir, E_2.tipo, tipo)$<br>$\text{genCode}(E.dir \ '=\ ' \alpha_1 \ '\leq \ ' \alpha_2)$                 |
|                                  | <b>Sino</b><br>error(tipos incompatibles)                                                                                                                                                                                                                                                                                                                              |
|                                  | <b>Fin Si</b>                                                                                                                                                                                                                                                                                                                                                          |
| $E \rightarrow E_1 \neq E_2$     | <b>Si</b> $\text{compatibles}(E_1.tipo, E_2.tipo)$ <b>Entonces</b>                                                                                                                                                                                                                                                                                                     |

|                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                      | E.dir = nuevaTemporal()<br>E.tipo = bool<br>tipo = max(E <sub>1</sub> .tipo, E <sub>2</sub> .tipo)<br>α <sub>1</sub> = ampliar(E <sub>1</sub> .dir, E <sub>1</sub> .tipo, tipo)<br>α <sub>2</sub> = ampliar(E <sub>2</sub> .dir, E <sub>2</sub> .tipo, tipo)<br>genCode(E.dir '=' α <sub>1</sub> '!' '=' α <sub>2</sub> )<br><b>Sino</b><br>error(tipos incompatibles)<br><b>Fin Si</b>                                                                                                                                                                       |
| E → E <sub>1</sub> == E <sub>2</sub> | <b>Si</b> compatibles(E <sub>1</sub> .tipo, E <sub>2</sub> .tipo) <b>Entonces</b><br>E.dir = nuevaTemporal()<br>E.tipo = bool<br>tipo = max(E <sub>1</sub> .tipo, E <sub>2</sub> .tipo)<br>α <sub>1</sub> = ampliar(E <sub>1</sub> .dir, E <sub>1</sub> .tipo, tipo)<br>α <sub>2</sub> = ampliar(E <sub>2</sub> .dir, E <sub>2</sub> .tipo, tipo)<br>genCode(E.dir '=' α <sub>1</sub> '==' α <sub>2</sub> )<br><b>Sino</b><br>error(tipos incompatibles)<br><b>Fin Si</b>                                                                                     |
| E → C                                | E.dir = nuevaTemporal()<br>E.tipo = C.tipo<br>genCode(E.dir '=' C.base '(' C.dir ')')                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| E → id                               | <b>Si</b> PilaTs.top().existe(id) <b>Entonces</b><br>E.tipo = PilaTs.top().getTipo(id)<br>E.dir = id<br><b>Sino Si</b> PilaTs.bottom().existe(id) <b>Entonces</b><br>E.tipo = PilaTs.bottom().getTipo(id)<br>E.dir = id<br><b>Sino</b><br>error("El id no fue declarado")<br><b>Fin Si</b>                                                                                                                                                                                                                                                                    |
| E → id( N )                          | <b>Si</b> PilaTs.bottom().existe(id) <b>Entonces</b><br><b>Si</b> PilaTs.bottom().getCat() = 'func' <b>Entonces</b><br><b>Si</b> PilaTs.bottom().getArgs() = N.lista <b>Entonces</b><br>E.dir = nuevaTemporal()<br>E.tipo = PilaTs.bottom().getTipo(id)<br>genCode(E.dir '=' 'call' id, N.lista.size)<br><b>Sino</b><br>error(La lista de parámetros no coincide con la lista de argumentos)<br><b>Fin Si</b><br><b>Sino</b><br>error(El identificador no es una función)<br><b>Fin Si</b><br><b>Sino</b><br>error("El id no fue declarado")<br><b>Fin Si</b> |
| E → Z                                | E.dir = nuevaTemporal()<br>E.tipo = Z.tipo<br>genCode(E.dir '=' Z.base '[' Z.dir ']')                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| E → intlit                           | E.tipo = tablaTipos.getId('int')<br>E.dir = num.lexema                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| E → floatlit                         | E.tipo = tablaTipos.getId('float')                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | E.dir = real.lexema                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| E → <b>true</b>          | E.tipo = tablaTipos.getId('bool')<br>E.dir = 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| E → <b>false</b>         | E.tipo = tablaTipos.getId('bool')<br>E.dir = 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| E → <b>charlit</b>       | E.tipo = tablaTipos.getId('char')<br>E.dir = charlit.lexema                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| N → M                    | N.lista = M.lista                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| N → $\varepsilon$        | N.lista = nulo                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| M → M <sub>1</sub> , E   | M.lista = M <sub>1</sub> .lista<br>M.lista.add(E.tipo)<br>genCode('param' E.dir)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| M → E                    | M.lista = nuevaLista()<br>M.lista.add(E.tipo)<br>genCode('param' E.dir)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| C → <b>id</b> [ E ]      | <b>Si</b> PilaTs.top().existe(id) <b>Entonces</b><br>tipo = PilaTs.top().getTipo(id)<br><b>Si</b> tablaTipos.getName(tipo) = 'array' <b>Entonces</b><br><b>Si</b> E.tipo = int <b>Entonces</b><br>C.base = id<br>C.tipo = tablaTipos.getTipoBase(tipo)<br>C.tam = tablaTipos.getTam(C.tipo)<br>C.dir = nuevaTemporal()<br>genCode( C.dir '=' E.dir * C.tam )<br><b>Sino</b><br>error("El índice de un arreglo debe ser entero")<br><b>Fin Si</b><br><b>Sino</b><br>error("El tipo del id no es un arreglo")<br><b>Fin Si</b><br><b>Sino Si</b> PilaTs.bottom().existe(id) <b>Entonces</b><br>tipo = PilaTs.bottom().getTipo(id)<br><b>Si</b> tablaTipos.getName(tipo) = 'array' <b>Entonces</b><br><b>Si</b> E.tipo = int <b>Entonces</b><br>C.base = id<br>C.tipo = tablaTipos.getTipoBase(tipo)<br>C.tam = tablaTipos.getTam(C.tipo)<br>C.dir = nuevaTemporal()<br>genCode( C.dir '=' E.dir * C.tam )<br><b>Sino</b><br>error("El índice de un arreglo debe ser entero")<br><b>Fin Si</b><br><b>Sino</b><br>error("El tipo del id no es un arreglo")<br><b>Fin Si</b><br><b>Sino</b><br>error("El id no fue declarado")<br><b>Fin Si</b> |
| C → C <sub>1</sub> [ E ] | <b>Si</b> tablaTipos.getName(C <sub>1</sub> .tipo) = 'array' <b>Entonces</b><br><b>Si</b> E.tipo = int <b>Entonces</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                          | <pre> C.base = C<sub>1</sub>.base C.tipo = tablaTipos.getTipoBase(C<sub>1</sub>.tipo) C.tam = tablaTipos.getTam(C.tipo) t = nuevaTemporal() C.dir = nuevaTemporal() genCode( t '=' E.dir '*' C.tam) genCode( C.dir '=' C<sub>1</sub>.dir '+' t ) </pre>                                                                                                                                                                                                                                                                         |
|                          | <pre> <b>Sino</b>     error("El índice de un arreglo debe ser entero") <b>Fin Si</b> </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|                          | <pre> <b>Sino</b>     error("El arreglo no puede ser indexada tantas veces") <b>Fin Si</b> </pre>                                                                                                                                                                                                                                                                                                                                                                                                                               |
| $Z \rightarrow Z_1 . id$ | <pre> <b>Si</b> tablaTipos.getName(Z<sub>1</sub>.tipo)='struct' <b>Entonces</b>     *ts = tablaTipos.getTS(Z<sub>1</sub>.tipo)     <b>Si</b> *ts.existe(id) <b>Entonces</b>         Z.dir = nuevaTemporal()         genCode(Z.dir '=' Z<sub>1</sub>.dir + ts.getDir(id) )         Z.base = Z<sub>1</sub>.base         Z.tipo = ts.getTipo(id) )     <b>Sino</b>         error("El id no fue declarado")     <b>Fin Si</b> <b>Sino</b>     error(No se puede usar el operador punto, el id no es un struct) <b>Fin Si</b> </pre> |
| $Z \rightarrow id$       | <pre> <b>Si</b> PilaTs.top().existe(id) <b>Entonces</b>     Z.tipo = PilaTs.top().getTipo(id)     Z.base = id     Z.dir = 0 <b>Sino Si</b> PilaTs.bottom().existe(id) <b>Entonces</b>     Z.tipo = PilaTs.bottom().getTipo(id)     Z.base = id     Z.dir = 0 <b>Sino</b>     error("El id no fue declarado") <b>Fin Si</b> </pre>                                                                                                                                                                                               |

Realizar el proceso de compilación para el siguiente programa:

```
struct X {
    int a;
    int b;
    float c;
}a, b;

float x, y;
int m, n;
int c;
int i, j;
float[5][7][3] w;
int z;
int w1;

def int suma(int x, int y){
    w1= 5;
    z = x + y + w1;
    return z;
}

def int main(){
    a.a= 8;
    a.b= 2;
    a.c = 1.5;
    m= 7;
    n = 15;
    c = suma(m, n);
    for(i=0; i < 5; i = i+1){
        for(j=0; j<7; j= j+1){
            for(k=0; k <3; k=k+1)
                a[i][j][k]= 0;
        }
    }

    for(i=0; i < 5; i = i+1){
        for(j=0; j<7; j= j+1){
            for(k=0; k <3; k=k+1)
                b[i][j][k]= b[i][j][k] + 3;
        }
    }
    return 0;
}
```

Mediante la definición dirigida por sintaxis, es decir; siguiendo las reglas gramaticales junto con las reglas semánticas genere los siguientes datos:

1. El grafo de dependencias
2. La pila de tablas de símbolos.
3. La tabla de tipos.
4. El código intermedio

**Sugerencia:** trabaje el árbol en una hoja doble carta.